PBMS API Documentation

Payroll & Benefits backend: app endpoints and the HR2, Finance and ESS integrations

Generated from the backend source at commit 1741ae0 · November 5, 2026 · 68 endpoints

Contents

1. Overview
2. Authentication
3. Roles and permissions
4. Endpoint index
5. Sign-in and account
6. User accounts
7. Employees
8. Dashboard and audit log
9. Synchronized app data
10. Payroll runs
11. Compensation
12. HMO benefits
13. Loans
14. Claims and reimbursement
15. Integration: HR2 attendance
16. Integration: Finance
17. Integration: ESS
18. Health checks and configuration

1. Overview

PBMS (Payroll & Benefits) is an Express 5 and PostgreSQL service in `backend-render/`. The React app calls it, and the HR2, Finance and ESS systems connect to it server to server.

Item	Value
Base URL	`https://<pbms-render-host>` (the frontend reads it from `VITE_API_BASE`). Paths have no version prefix.
Format	JSON (`Content-Type: application/json`), UTF-8, unless an endpoint says otherwise.
Request size	JSON bodies up to 5 MB. Funding receipts are raw file bodies up to 5 MB.
Dates	`YYYY-MM-DD`. Timestamps are ISO 8601 in UTC. "Today" for claims and HMO rules means Philippine time (Asia/Manila).
Money	Philippine pesos as numbers, rounded to 2 decimals.
CORS	`http://localhost:5173` and the origins listed in `ALLOWED_ORIGINS`.
List size	Most lists that read through the shared row helper (users, compensation, HMO, loans) stop at 1,000 rows and have no paging. `GET /hmo/history` has paging; `GET /audit-logs` returns the latest 2,000 logged entries plus older browser-recorded activity.

Response format

A successful response wraps its result in `data`. A failed response has `error` with a stable `code` and a readable `message`. `204 No Content` has no body. Exceptions: `GET /health` and `GET /api/health` have no wrapper, and funding receipts return the file itself.

```
// Success
{ "data": { "id": "c386ed75-..." } }

// Failure
{ "error": { "code": "VALIDATION_ERROR", "message": "employee_no is required." } }
```

Errors common to every endpoint

Status	Code	When
400	`VALIDATION_ERROR`	A required field is missing or invalid. The message names it.
400 / 409	`INVALID_REQUEST`	The database refused the values (invalid format, missing required column, duplicate or broken reference). 409 for duplicates and references. Applies to the routes under `/app-data`, `/employees`, `/audit-logs`, `/account`, `/payroll`, `/users`, `/compensation`, `/hmo`, `/loans`, `/government-loans` and `/integrations`.
401	`INVALID_SESSION`	The bearer token is missing, invalid or expired, its session was revoked, or the user is inactive.
401	`INVALID_API_KEY`	The `X-API-Key` header is missing or wrong (integration routes).
403	`FORBIDDEN`	The role lacks the permission, or a business rule refused the change.

Status	Code	When
413	`RECEIPT_TOO_LARGE` / `FILE_TOO_LARGE`	The body is over its limit. `RECEIPT_TOO_LARGE` on the route groups listed above, `FILE_TOO_LARGE` on all others.
500	`INTERNAL_ERROR`	An unexpected server error. A body that is not valid JSON also returns this.

2. Authentication

App users: bearer token

`POST /auth-login` returns a JWT valid for 8 hours. Send it on every app request. Each sign-in creates a session, listed in `GET /account`; signing out or revoking the session ends the token at once.

```
Authorization: Bearer <token>
```

Integrations: API key

HR2, Finance and ESS each have their own shared key, sent server to server. A key must be at least 32 characters; until it is set, that integration returns `503`. Generate one with `openssl rand -hex 32`, and never put a key in browser code.

```
X-API-Key: <shared key>
```

System	Environment variable	Routes	Not configured
HR2	`HR2_API_KEY`	`/integrations/hr2/*`	`HR2_NOT_CONFIGURED`
Finance	`FINANCE_API_KEY`	`/integrations/finance/*`	`FINANCE_NOT_CONFIGURED`
ESS	`ESS_API_KEY`	`/integrations/ess/*`	`ESS_NOT_CONFIGURED`

Passkey: confirming sensitive actions

Each user sets their own passkey in Settings. Generating and releasing payslips send it in the request; CSV exports and Payroll Settings changes confirm it first with `verify_passkey` on `POST /account`. It is 6 to 64 characters and must differ from the login password. Five wrong entries lock it for 15 minutes.

A wrong passkey on a payslip endpoint still returns `200`, with a `false` flag and a reason: `not_set`, `invalid` or `locked` (with `locked_until`). Check the flag, not only the status.

3. Roles and permissions

Endpoints require a permission from `backend-render/permissions.json`, the same map the frontend uses for its route guards. Role values are `admin`, `hr_staff` and `finance`.

Permission	Admin	HR Staff	Finance
`attendance.encode`	Yes	Yes	—
`attendance.view`	Yes	Yes	—

Permission	Admin	HR Staff	Finance
`audit.view`	Yes	Yes	—
`claims.approve`	Yes	Yes	—
`claims.encode`	Yes	Yes	—
`claims.pay`	Yes	Yes	—
`claims.view`	Yes	Yes	—
`compensation.approve`	Yes	—	—
`compensation.manage`	Yes	Yes	—
`compensation.view`	Yes	Yes	—
`dashboard.view`	Yes	Yes	Yes
`deductions.manage`	Yes	Yes	—
`deductions.view`	Yes	Yes	—
`employees.manage`	Yes	Yes	—
`employees.sensitive.view`	Yes	Yes	—
`employees.view`	Yes	Yes	Yes
`hmo.approve`	Yes	—	—
`hmo.manage`	Yes	Yes	—
`hmo.view`	Yes	Yes	—
`leave.approve`	Yes	Yes	—
`leave.encode`	Yes	Yes	—
`leave.view`	Yes	Yes	—
`loans.manage`	Yes	Yes	—
`payroll.approve`	Yes	Yes	Yes
`payroll.fund`	Yes	Yes	Yes
`payroll.generate`	Yes	Yes	—
`payroll.release`	Yes	Yes	—
`payroll.view`	Yes	Yes	Yes
`settings.edit`	Yes	—	—
`settings.view`	Yes	Yes	—
`users.manage`	Yes	—	—

Some rules apply on top of permissions: nobody approves a payroll run they prepared, a salary change or bonus they entered, or an HMO request they submitted. Nobody decides or pays their own claim (matched by email to their employee record), and whoever approved a claim cannot record its payment.

4. Endpoint index

All 68 endpoints, by module. Select a path to go to its details.

Sign-in and account

POST	`/auth-login`	Signs in and returns a token.
GET	`/current-user`	The signed-in user.
GET	`/account`	The account overview for Settings > Security.
POST	`/account`	Runs an account action, chosen by `action`.

User accounts

GET	`/users`	Lists user accounts.
POST	`/users`	Creates, updates or deletes an account. With no `action` it creates one.

Employees

GET	`/employees`	Lists all employees, ordered by employee number.
POST	`/employees`	Creates an employee and assigns the salary for their position when one exists.
PUT	`/employees/{id}`	Replaces an employee record.
DELETE	`/employees/{id}`	Deletes an employee and their HMO records. Returns `204`.

Dashboard and audit log

GET	`/dashboard`	Pending approvals and employee figures for the signed-in role.
GET	`/audit-logs`	Logs & Audit entries, newest first.
POST	`/audit-logs`	Records activity the app reports, such as an export.

Synchronized app data

GET	`/app-data`	Reads the keys the signed-in role may read.
POST	`/app-data`	Saves one key.
DELETE	`/app-data`	Deletes one key. Returns `204`.

Payroll runs

GET	`/payroll/runs/{id}/register`	The Payroll Register of a run, built from its saved figures.

POST	/payroll/runs/{id}/funding	Confirms the payroll account is funded; the run becomes Funded.
PUT	/payroll/runs/{id}/funding/receipt	Attaches or replaces the receipt while the run is Funded.
GET	/payroll/runs/{id}/funding/receipt	Returns the receipt file itself, never cached.
POST	/payroll/payslips/generate	Generates the payslips of a Paid run.
POST	/payroll/payslips/release	Releases payslips to ESS by department.
POST	/payroll/runs/{id}/ess-release	Releases every unreleased payslip of a Paid or Closed run to ESS.
DELETE	/payroll/runs/{id}	Deletes a run in any status, including approved and paid ones.
GET	/payroll/corrections	Lists all corrections, newest first.
POST	/payroll/corrections/{id}	Decides or completes a correction.
POST	/attendance/hr2/sync	Reads the attendance HR2 has sent so far and logs the check.

Compensation

GET	/compensation	Everything the Compensation pages show.
POST	/compensation	Runs a compensation action, chosen by `action`.

HMO benefits

GET	/hmo/{resource}	Lists the records of a resource.
GET	/hmo/{resource}/{id}	Same as the list: the id is ignored on reads, so filter the result yourself.
POST	/hmo/{resource}	Creates a record, or runs an action when the body has `action`.
POST	/hmo/{resource}/{id}	Runs an action on the record in the path.
PATCH	/hmo/{resource}/{id}	Runs an action, or updates a provider. `PUT` behaves the same.
PUT	/hmo/{resource}/{id}	Same as `PATCH`.
DELETE	/hmo/{resource}/{id}	Always refused: `409 HMO_DELETE_NOT_ALLOWED`.

Loans

GET	/loans	All loan records and their transactions.
GET	/loans/schedule	Repayments due in a cutoff.
POST	/loans	Records a cash advance as Pending. Returns `201`.

PATCH	`/loans/{id}`	Approves or rejects a Pending record.
POST	`/loans/settle`	Posts the repayments of a Paid run once.
GET	`/government-loans`	All government loans and their deduction transactions.
GET	`/government-loans/schedule`	Active loans payroll may deduct in a cutoff.
GET	`/government-loans/{id}/notice`	The attached agency notice.
POST	`/government-loans`	Encodes a loan as Pending. Returns `201`.
PATCH	`/government-loans/{id}`	Edits, verifies or rejects a Pending loan.
POST	`/government-loans/settle`	Posts the deductions of a Paid run once; a loan reaching zero closes.

Claims and reimbursement

GET	`/claims/totals`	Claim totals by month, type and status, with no employee or receipt details.
GET	`/claims/{id}/receipt`	A claim's receipt file.
GET	`/claims/{id}/payment-proof`	A claim's proof of payment, or its batch's proof.
POST	`/claims/{id}/payment`	Records the reimbursement of one approved claim.
POST	`/claims/batches`	Creates a batch of approved claims paid by one bank transfer. Returns `201`.
POST	`/claims/batches/{id}/payment`	Marks a batch paid after the bank transfer.
POST	`/claims/batches/{id}/cancel`	Cancels an unpaid batch; its claims return to the queue.

Integration: HR2 attendance

POST	`/integrations/hr2/attendance`	Receives attendance records.

Integration: Finance

GET	`/integrations/finance/payroll-runs`	Lists submitted runs, newest first.
GET	`/integrations/finance/payroll-runs/{id}`	One run with its employees.
GET	`/integrations/finance/payroll-runs/{id}/funding-receipt`	The funding receipt file (JPG, PNG or PDF).
POST	`/integrations/finance/payroll-runs/{id}/decision`	Removed: always returns `410 DECISIONS_IN_PBMS`.

Integration: ESS

POST	`/integrations/ess/claims`	Submits a claim. Returns `201`, or `200` for a repeated `ess_reference`.

PUT	`/integrations/ess/claims/{claimNumber}`	Resubmits a Returned claim, with all fields again. It goes back to Pending Approval.
GET	`/integrations/ess/claims`	An employee's claims, newest first.
GET	`/integrations/ess/payslips`	An employee's released payslips, newest release first.
GET	`/integrations/ess/notifications`	An employee's latest 100 notifications, newest first.
POST	`/integrations/ess/corrections`	Requests a payroll correction. Returns `201`, or `200` for a repeated `ess_reference`.
GET	`/integrations/ess/corrections`	An employee's corrections, newest first.

Health checks and configuration

GET	`/health`	Confirms the service is running.
GET	`/api/health`	Row counts, to check which database the service reads.

5. Sign-in and account

POST `/auth-login`

Signs in and returns a token.

Access: Public

REQUEST BODY

Field	Required	Notes
`email`	Yes	Not case-sensitive.
`password`	Yes	

RESPONSE

```
{
  "data": {
    "token": "<jwt>",
    "user": {
      "id": "uuid",
      "employeeId": "USR-0001",
      "name": "Maria Santos",
      "displayName": "Maria",
      "role": "HR Staff"
    }
  }
}
```

`user.role` here is the label: `Admin`, `HR Staff` or `Finance`.

ERRORS

Status	Code	When
401	`INVALID_CREDENTIALS`	Wrong email or password, or an inactive account.

GET `/current-user`

The signed-in user.

Access: Bearer

RESPONSE

```
{
  "data": [
    {
      "id": "uuid",
      "name": "Maria Santos",
      "display_name": "Maria",
      "email": "maria@example.com",
      "employee_id": "USR-0001",
      "role": "hr_staff"
    }
  ]
}
```

`data` is a list with one item.

GET `/account`

The account overview for Settings > Security.

Access: Bearer

RESPONSE

```
{
  "data": {
    "otp_enabled": true,
    "otp_available": false,
    "passkey_set": true,
    "passkey_locked_until": null,
    "current_session_id": "uuid",
    "sessions": [
      {
        "id": "uuid",
        "device": "Windows PC",
        "browser": "Edge",
        "created_at": "2026-11-05T01:00:00.000Z",
        "last_active_at": "2026-11-05T01:20:00.000Z",
        "current": true
      }
    ],
    "history": [
      {
        "id": "uuid",
        "device": "Windows PC",
        "browser": "Edge",
        "created_at": "2026-11-05T01:00:00.000Z",
        "last_active_at": "2026-11-05T01:20:00.000Z",
```

```
        "ended_at": null,
        "status": "Active",
        "current": true
      }
    ]
  }
}
```

`history` holds the latest 100 sign-ins; `status` is `Active`, `Signed out` or `Expired`. OTP is turned off on this server, so `otp_available` is always `false`.

POST `/account`

Runs an account action, chosen by `action`.

Access: Bearer (own account only)

Actions marked "password" also need `current_password`; a wrong one returns `400 INVALID_PASSWORD`. Actions that return "overview" reply with the same object as `GET /account`.

ERRORS

Status	Code	When
400	`UNKNOWN_ACTION`	The action is not supported.
404	`USER_NOT_FOUND`	The account no longer exists.

action	Fields	Returns	Specific errors
`sign_out`	—	`{ signed_out: true }`	—
`set_display_name`	`display_name` (up to 80 characters)	`{ display_name }`	—
`verify_passkey`	`purpose` (`export`, `release` or `settings`), `passkey`, `detail` (optional)	`{ verified, reason?, locked_until?, attempts_left? }`	`INVALID_PURPOSE`
`change_passkey`	`current_passkey`, `new_passkey`	overview	`INVALID_PASSKEY`, `PASSKEY_NOT_SET` (409), `PASSKEY_LOCKED` (423), `PASSKEY_UNCHANGED`, `WEAK_PASSKEY`, `PASSKEY_MATCHES_PASSWORD`
`set_passkey`	password, `new_passkey`	overview	`PASSKEY_ALREADY_SET` (409), `WEAK_PASSKEY`, `PASSKEY_MATCHES_PASSWORD`
`reset_passkey`	password, `new_passkey`	overview	`WEAK_PASSKEY`, `PASSKEY_MATCHES_PASSWORD`
`change_password`	password, `new_password`	overview; signs out other sessions	`WEAK_PASSWORD`, `PASSWORD_UNCHANGED`
`set_otp`	password, `enabled` (boolean)	overview	`INVALID_OTP_SETTING`

action	Fields	Returns	Specific errors
`revoke_session`	password, `session_id`	overview	`CURRENT_SESSION`, `SESSION_NOT_FOUND` (404)
`revoke_other_sessions`	password	overview	—

A new password needs 8 to 128 characters with upper-case and lower-case letters, a number and a symbol.

6. User accounts

GET `/users`

Lists user accounts.
Access: `users.manage`

RESPONSE

```
{
  "data": [
    {
      "id": "uuid",
      "employee_id": "USR-0001",
      "name": "Maria Santos",
      "display_name": "Maria",
      "email": "maria@example.com",
      "role": "hr_staff",
      "status": "active"
    }
  ]
}
```

POST `/users`

Creates, updates or deletes an account. With no `action` it creates one.
Access: `users.manage`

Account IDs (`USR-0001`, ...) are assigned by the server. Every change is written to Logs & Audit.

ERRORS

Status	Code	When
400	`INVALID_EMAIL`, `WEAK_PASSWORD`, `INVALID_ROLE`, `INVALID_STATUS`	A field is invalid.
400	`CANNOT_DEACTIVATE_SELF`, `CANNOT_DELETE_SELF`	A user cannot do this to their own account.
404	`USER_NOT_FOUND`	No account with that `id`.
409	`DUPLICATE_EMAIL`	The email is already used.
409	`LAST_ADMIN`	The change would leave no active Admin.

action	Fields	Returns
(none)	`name`, `display_name`, `email`, `password` (at least 8 characters), `role` (default `hr_staff`)	`201 { user, otp_sent: false, otp_disabled: true, challenge_id: null, emailId: null }`
`update`	`id`, `name`, `email`, `role` (optional)	`{ user }`
`update_role`	`id`, `role`	`{ user }`
`set_status`	`id`, `status` (`active` or `inactive`)	`{ user }`
`delete`	`id`	`{ deleted: true, id }`

7. Employees

GET `/employees`

Lists all employees, ordered by employee number.

Access: `employees.view`

RESPONSE


```
{
  "data": [
    {
      "id": "uuid",
      "employee_number": "EMP-0001",
      "first_name": "Juan",
      "middle_name": null,
      "last_name": "Dela Cruz",
      "email": "juan@example.com",
      "position": "Site Engineer",
      "department": "Engineering",
      "hire_date": "2025-02-01",
      "employment_status": "Regular",
      "status": "Active",
      "basic_salary": "30000",
      "position_rate": "1363.64",
      "phone": null,
      "address": null,
      "birth_date": null,
      "sss_number": null,
      "philhealth_number": null,
      "pagibig_number": null,
      "tin": null,
      "bank_name": null,
      "bank_account_number": null,
      "salary_grade_id": null,
      "updated_at": null,
      "separated_on": null
    }
  ]
}
```


`sss_number`, `philhealth_number`, `pagibig_number`, `tin` and `bank_account_number` are left out for roles without `employees.sensitive.view`. This route returns numeric columns as strings.

POST `/employees`

Creates an employee and assigns the salary for their position when one exists.

Access: `employees.manage`

REQUEST BODY

Field	Required	Notes
`employee_number`, `first_name`, `last_name`	Yes	`employee_number` must be unique.
`middle_name`, `email`, `position`, `department`, `hire_date`, `employment_status`	No	`employment_status` classifications: `Regular`, `Probationary`, `Part-Time`, `Contractual`, `Project-Based`, `Temporary`, or `Resigned`.
`status`	No	Default `Active`. Others: `Inactive`, `On Leave`.
`phone`, `address`, `birth_date`, `sss_number`, `philhealth_number`, `pagibig_number`, `tin`, `bank_name`, `bank_account_number`	No	Blank values are saved as null; at most 300 characters.

RESPONSE

```
{
  "data": {
    "id": "uuid",
    "employee_number": "EMP-0001",
    "salary_assigned": true
  }
}
```

`201 Created` with the full employee record plus `salary_assigned`.

ERRORS

Status	Code	When
409	`DUPLICATE`	The employee number already exists.

PUT `/employees/{id}`

Replaces an employee record.

Access: `employees.manage`

Send every field, as for creating one: a field left out is cleared. A required field left out fails with `400 INVALID_REQUEST`, and `status` falls back to `Active`. Roles without `employees.sensitive.view` keep the stored sensitive values. The day an employee becomes inactive or resigned is recorded in `separated_on`.

RESPONSE

```
{
  "data": {
    "id": "uuid",
    "salary_assigned": false
  }
}
```

ERRORS

Status	Code	When
404	`NOT_FOUND`	No employee with that id.

DELETE `/employees/{id}`

Deletes an employee and their HMO records. Returns `204`.

Access: `employees.manage`

Also returns `204` when the id does not exist.

8. Dashboard and audit log

GET `/dashboard`

Pending approvals and employee figures for the signed-in role.

Access: `dashboard.view`

RESPONSE


```
{
  "data": {
    "approvals": [
      {
        "key": "claims",
        "label": "Claims",
        "count": 3,
        "detail": "3 claims to verify",
        "action": "Review",
        "link": "/claims-reimbursement/claims?status=pending"
      }
    ],
    "departments": {
      "total": 42,
      "slices": [
        {
          "name": "Engineering",
          "count": 20,
          "percent": 48
        }
      ]
    },
    "employeeOverview": {
      "total": 45,
      "recordStatus": [
        {
          "name": "Active",
          "count": 42
        }
      ],
      "employmentType": [
        {
          "name": "Regular",
          "count": 30
        }
      ]
```

```
    }
  }
}
```

`approvals` keys: `claims`, `payroll-approval`, `payroll-payment`, `hmo`; each appears only when the role has the matching permission. `departments` lists the five largest departments, with `Other` for the rest. `departments` and `employeeOverview` are `null` without `employees.view`.

GET /audit-logs

Logs & Audit entries, newest first.

Access: `audit.view`

RESPONSE

```
{
  "data": [
    {
      "id": "uuid",
      "timestamp": "2026-11-05T01:00:00.000Z",
      "user": "Maria Santos",
      "role": "HR Staff",
      "action": "Submitted payroll",
      "module": "Payroll",
      "description": "Submitted payroll for 2026-10-01 to 2026-10-15 for approval (42 employees, net
₱1,039,499.50).",
      "ipDevice": "203.0.113.5 · Edge"
    }
  ]
}
```

POST /audit-logs

Records activity the app reports, such as an export.

Access: Bearer (any role)

REQUEST BODY

Field	Required	Notes
`action`	Yes	Up to 120 characters.
`module`	No	Up to 80 characters.
`description`	No	Up to 1,000 characters.

RESPONSE

```
{
  "data": {
    "recorded": true
  }
}
```

`201 Created`.

9. Synchronized app data

Payroll runs, claims, leave, attendance, settings and notifications are saved as whole values under named keys. The server checks each write against the role and the workflow rules, whatever the browser sends, and logs the state changes it allows.

GET `/app-data`

Reads the keys the signed-in role may read.

Access: Bearer (rows follow the table below)

QUERY PARAMETERS

Name	Required	Notes
`scope`	No	`payroll-rules` returns only the rates and rules payroll needs, read-only, and requires `payroll.view`.

RESPONSE

```
{
  "data": [
    {
      "key": "payroll:runs",
      "value": [],
      "deleted": false
    }
  ]
}
```

With `scope=payroll-rules` each item has only `key` and `value`.

POST `/app-data`

Saves one key.

Access: Bearer (rules below)

REQUEST BODY

Field	Required	Notes
`key`	Yes	At most 200 characters, starting with `admin:`, `audit:`, `attendance:`, `benefits:`, `claims:`, `compensation:`, `deductions:`, `leave:`, `notifications:`, `payroll:`, `reimbursement:` or `settings:`.
`value`	Unless deleting	The whole new value.
`deleted`	No	`true` deletes the key.

RESPONSE

```
{
  "data": {
    "key": "payroll:runs",
    "value": [],
    "deleted": false
```

```
  }
}
```

The returned `value` is what was stored; the server may restore fields it controls, so use it instead of what was sent.

ERRORS

Status	Code	When
400	`INVALID_DATA_KEY`	The key is not allowed.
403	`FORBIDDEN`	The role or a workflow rule refused the write. The message says why.
409	`INVALID_TRANSITION`	A payroll status change outside the lifecycle.
409	`STRICT_MODE`	Strict Mode blocks the new run: timesheet not locked, HR2 not synced to the cutoff end, or a run already exists for the cutoff.

DELETE `/app-data`

Deletes one key. Returns `204`.
Access: Bearer (rules below)

QUERY PARAMETERS

Name	Required	Notes
`key`	Yes	The key to delete. `payroll:runs` and `claims:records` cannot be deleted.

ERRORS

Status	Code	When
400	`INVALID_DATA_KEY`	The key is not allowed.

Keys and rules

Key	Read	Write	Rules
`payroll:runs`	Any role	By step	See Payroll runs. Runs missing from a save are kept.
`payroll:payouts`	Any role	`payroll.generate`	Each employee's payout method and account.
`payroll:records`	Any role	`payroll.generate` or `payroll.approve`	
`payroll:timesheet-locks`	Any role	`payroll.generate`	Locks are only added, and only once HR2 attendance reaches the cutoff end.
`payroll:bank-results`	Any role	`payroll.release`	Results are only added; each needs its run and cutoff dates.
Other `payroll:*`, `compensation:thirteenth-month-rules`	`settings.view`	`settings.edit`	A withholding tax version used by a run cannot be changed or removed.

Key	Read	Write	Rules
`settings:*`	Any role	`settings.edit`	Company, regional, branding, privacy and backup settings.
`notifications:records`	Any role	Any role	New ones must be unread. A user may only mark their own role's copies read; unread ones cannot be removed.
`claims:records`	`claims.view`	`claims.encode`, `claims.approve`	Decisions only: approve, return or reject. Claims come from ESS; details and payments cannot be changed here.
`reimbursement:batches`	`claims.view`	Server only	Changes only through the batch endpoints.
Other `claims:*`, `reimbursement:*`	`claims.view`	`claims.encode`	
`attendance:hr2`	Any role	Server only	Received only from HR2.
Other `attendance:*`	Any role	`attendance.encode`	Saved records are locked; writes may only add.
`leave:records`	Any role	`leave.encode`, `leave.approve`	Starts `Pending`; a decided leave is locked.
Other `leave:*`	Any role	`leave.encode`	
`deductions:other`	Any role	`deductions.manage`	New ones must be active and authorized, and can only be stopped afterwards.
Other `deductions:*`	Any role	`deductions.manage`	
Other `compensation:*`	Any role	`compensation.manage`	
`benefits:*`	Any role	`hmo.manage`	
`admin:users`	`users.manage`	`users.manage`	
`audit:*`	Nobody	Nobody	Use `/audit-logs`.

10. Payroll runs

Runs are stored in the `payroll:runs` key. HR creates and moves them with `POST /app-data`, sending the whole list; funding, payslips, release and deletion have their own endpoints.

Lifecycle

From	To	How	Permission	Conditions
(new)	Draft	Save	`payroll.generate`	Prepared by the signed-in user. `payDate` must be the automatic date for the cutoff.
Draft, Computed, Returned	Computed	Save	`payroll.generate`	Each employee has a Pending payout line, and gross, deductions and net follow the register formulas.
Computed	Pending Approval	Save	`payroll.generate`	Contents unchanged.
Pending Approval	Approved, Returned, Rejected	Save	`payroll.approve`	Not the preparer. Returned and Rejected need a comment of at least 3 characters. Rejected is locked.
Approved	Funded	`POST .../funding`	`payroll.fund`	Optional step: payouts can also be exported directly from Approved.
Approved, Funded	Exported	Save	`payroll.release`	Comes with the first payout file.
Exported	Paid	Save	`payroll.release`	Every employee is Paid and `paidAt` is set.
Paid	Closed	Release endpoints	`payroll.release` + passkey	Set by the server when every payslip is released.

Payout line statuses: Pending, then Exported, then Paid or Failed (with a reason of at least 3 characters). A Failed line returns to Pending after its details are fixed; a Paid line is locked. Methods are `Bank`, `E-wallet` and `Cash`.

Run object

Field	Notes
`id`, `runType`, `from`, `to`, `payDate`, `createdAt`, `preparedBy`, `preparedById`	Fixed at creation. `runType` is `regular` or `thirteenth-month`.
`status`	Draft, Computed, Pending Approval, Returned, Rejected, Approved, Funded, Exported, Paid or Closed.
`rows`	One per employee: `input` (`empId`, `name` and pay inputs), `result` (`gross`, `deductions`, `net`, `tax`, ...), `hmo`, `position`, `governmentLoans`, `loanRepayments`.
`payouts`, `exports`	Payout line per employee number (`method`, `status`, `reason`, `exportId`, account details) and the exported files, which can only be added.
`submittedAt`, `submittedBy`, `decidedBy`, `decidedById`, `decidedAt`, `decisionComment`, `paidAt`, `paidBy`	Workflow details.

Field	Notes
`funding`	Server only: `amount`, `confirmedAt`, `confirmedBy`, and for a receipt `receiptId`, `fileName`, `mimeType`, `size`, `uploadedAt`, `uploadedBy`.
`payslipsGeneratedAt`, `payslipsGeneratedBy`, `released`, `essReleased`, `closedAt`	Server only. `released` is by department, `essReleased` by employee number.

Endpoints

GET `/payroll/runs/{id}/register`

The Payroll Register of a run, built from its saved figures.

Access: `payroll.view`

RESPONSE


```
{
  "data": {
    "company": "Archon Nell Inc.",
    "runId": "PR-5F1C0A2B",
    "cutoff": "Oct 1 – 15, 2026",
    "payDate": "Oct 15, 2026",
    "columns": [
      {
        "key": "basic",
        "label": "Basic pay"
      }
    ],
    "lines": [
      {
        "empId": "EMP-0001",
        "name": "Ana Reyes",
        "position": "Site Engineer",
        "basic": 15000,
        "allowance": 500,
        "bonuses": 0,
        "overtime": 360.58,
        "holidayLeave": 0,
        "absences": 0,
        "lateUt": 144.23,
        "sss": 750,
        "philhealth": 375,
        "pagibig": 100,
        "hmo": 250,
        "tax": 611.15,
        "loans": 0,
        "other": 0,
        "gross": 15860.58,
        "totalDeductions": 2230.38,
        "net": 13630.2
      }
    ],
    "totals": {
      "gross": 15860.58,
      "net": 13630.2
    }
```

```
  }
}
```

Gross = basic + allowance + bonuses + overtime + holidayLeave. Total deductions = absences + lateUt + sss + philhealth + pagibig + hmo + tax + loans + other. Net = gross - total deductions. The Tax column appears only when the Payroll Run Options turn it on.

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`	No run with that id.

POST `/payroll/runs/{id}/funding`

Confirms the payroll account is funded; the run becomes Funded.

Access: `payroll.fund`

The body is the raw receipt file, or empty to attach the receipt later. Set `Content-Type` to the file type and `X-File-Name` to the URI-encoded file name. JPG, PNG or PDF, at most 5 MB; the type is checked from the file content.

RESPONSE

```
{
  "data": {
    "runs": [
      "<the full updated list of runs>"
    ]
  }
}
```

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`	No run with that id.
409	`PAYROLL_NOT_APPROVED`	The run is not Approved.
413	`RECEIPT_TOO_LARGE`	Over 5 MB.
415	`RECEIPT_TYPE_NOT_ALLOWED`	Not a JPG, PNG or PDF, or the type does not match the content.

PUT `/payroll/runs/{id}/funding/receipt`

Attaches or replaces the receipt while the run is Funded.

Access: `payroll.fund`

Same file format as above; the file is required. A replaced file is deleted.

RESPONSE

```
{
  "data": {
    "runs": [
      "<the full updated list of runs>"
    ]
  }
}
```

ERRORS

Status	Code	When
400	`RECEIPT_REQUIRED`	No file was sent.
409	`RECEIPT_LOCKED`	The run is no longer Funded.

GET /payroll/runs/{id}/funding/receipt

Returns the receipt file itself, never cached.

Access: `payroll.fund`

QUERY PARAMETERS

Name	Required	Notes
`download`	No	Any value downloads it instead of showing it inline.

ERRORS

Status	Code	When
404	`RECEIPT_NOT_FOUND`	The run has no receipt.

POST /payroll/payslips/generate

Generates the payslips of a Paid run.

Access: `payroll.release` + passkey

REQUEST BODY

Field	Required	Notes
`run_id`	Yes	
`passkey`	Yes	The signed-in user's passkey.

RESPONSE

```
// Generated
{ "data": { "generated": true, "runs": ["<the full updated list of runs>"] } }

// Passkey refused (still 200)
{ "data": { "generated": false, "reason": "invalid" } }
```

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`	No run with that id.
409	`PAYROLL_NOT_PAID`	The run is not Paid.
409	`PAYSLIPS_ALREADY_GENERATED`	Already generated.

POST /payroll/payslips/release

Releases payslips to ESS by department.

Access: `payroll.release` + passkey

Each employee of those departments gets their payslip in ESS and a notification. The run closes when every payslip is released.

REQUEST BODY

Field	Required	Notes
`run_id`	Yes	
`departments`	Yes	List of department names. Employees with no department are under `Unassigned`.
`passkey`	Yes	
`all`	No	`true` when this is Release All; used in the audit entry.

RESPONSE

```
// Released
{ "data": { "released": true, "runs": ["<the full updated list of runs>"] } }

// Passkey refused (still 200)
{ "data": { "released": false, "reason": "locked", "locked_until": "2026-11-05T01:15:00.000Z" } }
```

ERRORS

Status	Code	When
400	`VALIDATION_ERROR`	`departments` is empty.
409	`PAYROLL_NOT_PAID`	The run is not Paid.
409	`PAYSLIPS_ALREADY_RELEASED`	Those departments are already released.

POST /payroll/runs/{id}/ess-release

Releases every unreleased payslip of a Paid or Closed run to ESS.

Access: `payroll.release` + passkey

Releasing again sends only what is still unreleased, so nothing is sent twice.

REQUEST BODY

Field	Required	Notes
`passkey`	Yes	

RESPONSE

```
// Verified
{ "data": { "verified": true, "sent": 40, "released": 42, "total": 42, "runs": ["<the full list of
runs>"] } }

// Passkey refused (still 200)
{ "data": { "verified": false, "reason": "invalid" } }
```

ERRORS

Status	Code	When
409	`RELEASE_NOT_ALLOWED`	The run is not Paid or Closed.

DELETE /payroll/runs/{id}

Deletes a run in any status, including approved and paid ones.

Access: `payroll.generate` or `payroll.approve`

Its funding receipt, ESS payslips and payslip notifications go with it. The audit entry keeps its status and net pay.

RESPONSE

```
{
  "data": {
    "runs": [
      "<the remaining runs>"
    ]
  }
}
```

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`	No run with that id.

Payroll corrections

Employees request corrections from ESS. An approved correction is paid outside the system and is never added to a later run. Statuses: `Pending`, `Approved`, `Rejected`, `Completed`.

GET /payroll/corrections

Lists all corrections, newest first.

Access: `payroll.view`

Each item has the shape shown under `GET /integrations/ess/corrections`.

POST /payroll/corrections/{id}

Decides or completes a correction.

Access: `payroll.approve` (approve, reject) or `payroll.release` (complete)

REQUEST BODY

Field	Required	Notes
`action`	Yes	`approve`, `reject` or `complete`. Only a Pending one can be approved or rejected; only an Approved one can be completed.
`comment`	To reject	At least 3 characters when rejecting; up to 500. Optional otherwise.

Returns the correction.

ERRORS

Status	Code	When
404	`CORRECTION_NOT_FOUND`	No correction with that id.
409	`CORRECTION_NOT_ALLOWED`	The action does not fit its status, the action is unknown, or the comment is missing.

Attendance

POST `/attendance/hr2/sync`

Reads the attendance HR2 has sent so far and logs the check.

Access: `attendance.view`

HR2 pushes attendance to PBMS; this does not call HR2.

RESPONSE

```
{
  "data": {
    "syncedThrough": "2026-10-15",
    "receivedAt": "2026-10-15T08:00:00.000Z",
    "records": 2240
  }
}
```

11. Compensation

GET `/compensation`

Everything the Compensation pages show.

Access: `compensation.view`

RESPONSE

```
{
  "data": {
    "salary_grades": [],
    "assignments": [],
    "proposals": [],
    "history": [],
    "rewards": [],
    "legacy_import_complete": true,
    "legacy_import_counts": null
  }
}
```

POST `/compensation`

Runs a compensation action, chosen by `action`.

Access: `compensation.approve` for reviews; `compensation.manage` for the rest

ERRORS

Status	Code	When
400	`UNKNOWN_ACTION`	The action is not supported.
400	`INVALID_AMOUNT`, `INVALID_DATE`, `INVALID_ID`, `INVALID_DECISION`	A field is invalid.
403	`FORBIDDEN`	Reviewing an entry the user created.

action	Fields	Specific errors
`create_grade`	`position`, `basic_salary`, `status` (`Active` default, or `Inactive`). The code (`SG-001`, ...) is generated.	`INVALID_GRADE_STATUS`, `SALARY_GRADE_GENERATOR_NOT_READY` (503), `SALARY_GRADE_CODE_GENERATION_FAILED` (500), `LEGACY_COMPENSATION_IMPORT_REQUIRED` (409)
`update_grade`	`id`, `position`, `basic_salary`	`SALARY_GRADE_NOT_FOUND` (404)
`delete_grade`	`id`	`SALARY_GRADE_IN_USE` (409)
`create_initial_assignment`	`employee_id`, `basic_salary`, `effective_date`, `salary_code` (optional)	`EMPLOYEE_NOT_FOUND` (404), `COMPENSATION_ALREADY_ASSIGNED` (409), `INVALID_SALARY_GRADE`
`create_proposal`	`employee_id`, `proposal_type` (`Salary Adjustment`, `Merit Increase`, `Promotion Increase`), `effective_date`, `salary_code` or `new_salary`, `notes` (optional)	`INVALID_PROPOSAL_TYPE`, `COMPENSATION_NOT_ASSIGNED` (409), `UNCHANGED_SALARY`
`review_proposal`	`id`, `decision` (`approve` or `reject`), `rejection_reason` (to reject)	`PROPOSAL_NOT_FOUND` (404), `PROPOSAL_ALREADY_REVIEWED` (409)
`create_reward`	`employee_id`, `reward_type` (`Allowance`, `Incentive`, `Bonus`), and the fields below	`INVALID_REWARD_TYPE`, `INVALID_FREQUENCY`, `INVALID_ALLOWANCE_STATUS`, `INVALID_TAXABLE`, `INVALID_CUTOFF`, `INVALID_TAX_CATEGORY`
`update_reward`	`id` plus the same fields. An edited bonus or incentive returns to Pending. An allowance cannot become a bonus.	`REWARD_NOT_FOUND` (404)
`delete_reward`	`id`	`REWARD_NOT_FOUND` (404)
`review_reward`	`id`, `decision` (default `approve`). Bonuses and incentives only.	`REWARD_NOT_REVIEWABLE`, `REWARD_ALREADY_REVIEWED` (409)
`approve_reward`	`id`. Same as `review_reward` with `approve`.	As above

action	Fields	Specific errors
`import_legacy`	`legacy_data`, `retry_failed` (optional). One-time import of browser-stored compensation.	`LEGACY_SALARY_GRADE_CODE_COLLISION` (409), `LEGACY_SALARY_GRADE_ID_COLLISION` (409)

Reward type	Fields
Allowance	`amount`, `allowance_type`, `frequency` (`Per Cutoff` or `Monthly`), `status` (`Active` or `Inactive`), `taxable` (boolean), `effective_date`, `notes` (optional)
Bonus, Incentive	`amount`, `cutoff_start`, `cutoff_end`, `notes`, and `tax_category`: `christmas`, `productivity`, `loyalty`, `gift` or `similar` (count toward the ₱90,000 exemption), `commission` or `taxable`. Older clients may send `taxable` instead. Starts Pending.

12. HMO benefits

Maxicare is the only provider, and it is not connected to PBMS: HR records what was sent to Maxicare and what Maxicare answered. `{resource}` is `providers`, `plans`, `enrollments`, `dependents` or `history`; any other value returns `404 NOT_FOUND`.

Enrollment flow: submitted, approved or rejected by a different user, sent to Maxicare, then confirmed (Active) with the member ID or marked not accepted. A not-accepted request can be corrected and sent again. Plans, enrollments and dependents expire by date, checked when they are read.

GET `/hmo/{resource}`

Lists the records of a resource.
Access: `hmo.view`

QUERY PARAMETERS

Name	Required	Notes
`page`, `page_size`	No	History only. Defaults 1 and 25; `page_size` at most 100.
`entity_type`, `entity_id`, `search`	No	History only. Filters; `search` looks in the whole entry.

Enrollments also include `employee`, `employee_eligibility_date`, `employee_eligible_for_effective_date` and `maxicare_answered_by_name`; dependents include `maxicare_answered_by_name`. History returns `{ items, page, page_size, total }`.

GET `/hmo/{resource}/{id}`

Same as the list: the id is ignored on reads, so filter the result yourself.
Access: `hmo.view`

POST `/hmo/{resource}`

Creates a record, or runs an action when the body has `action`.
Access: `hmo.manage`; `hmo.approve` to approve or reject

Without `action` it creates a record and returns `201` with it. With `action`, send the record `id` in the body or path.

Create	Required	Optional	Rules
providers	`name` (must be Maxicare)	`contact_person`, `contact_number`, `status`	One active provider.
plans	`provider_id`, `name`, `coverage_start`, `coverage_end`	`annual_benefit_limit`, `inpatient`, `outpatient`, `emergency`, `preventive_care`, `dental`, `employee_monthly_premium`, `dependent_monthly_premium`, `dependent_company_share`, `dependent_member_share`, `collect_employee_share_via_payroll`, `status` (default `Draft`)	The company pays 100% of the employee premium. Dependent shares add up to 100%.
enrollments	`plan_id`, `employee_id`, `effective_date`	`expiration_date`	Active employee with 6 months of service by the effective date, no other open enrollment, and dates inside an active plan.
dependents	`enrollment_id`, `name`, `relationship` (`Spouse` or `Child`)	`date_of_birth`, `effective_date`, `expiration_date`, `company_sponsored`	Enrollment must be Active. One company-sponsored dependent per employee; the first is by default.

action	Applies to	Fields	Result
`update`	plans	Any plan field; `apply_to_dependents` (boolean) applies a new premium or split to current dependents	Plan updated; past runs keep their amounts.
`approve`	enrollments, dependents	—	Needs `hmo.approve`; not the submitter.
`reject`	enrollments, dependents	`rejection_reason`	Needs `hmo.approve`; not the submitter. Closes the request.
`cancel`	enrollments, dependents	—	Only the submitter, while pending. Closes it as Rejected.
`submit_to_maxicare`	enrollments, dependents	—	After approval: `maxicare_status` becomes Pending.
`maxicare_reject`	enrollments, dependents	`rejection_reason`	Marks it not accepted; HR and Admin are notified.
`activate`	enrollments, dependents	`maxicare_reference`, `allow_shared` (optional)	Confirms it with the member ID; becomes Active.

action	Applies to	Fields	Result
`correct_member_id`	enrollments, dependents	`maxicare_reference`, `reason`, `allow_shared` (optional)	Corrects a confirmed ID; the old one stays in history.
`terminate`	enrollments, dependents	—	Ends coverage. Ending an enrollment ends its dependents.

Common HMO error codes. 400: `MAXICARE_REQUIRED`, `INVALID_HMO_DATE`, `INVALID_HMO_DATES`, `INVALID_EMPLOYEE_CONTRIBUTION_POLICY`, `INVALID_DEPENDENT_CONTRIBUTION_POLICY`, `INVALID_HMO_PLAN_STATUS`, `INELIGIBLE_DEPENDENT_RELATIONSHIP`, `MAXICARE_REFERENCE_REQUIRED`, `MAXICARE_REFERENCE_UNCHANGED`, `HMO_RECORD_ID_REQUIRED`, `HMO_PLAN_ID_REQUIRED`, `HMO_ACTION_NOT_SUPPORTED`. 403: `HMO_NOT_SUBMITTER`. 404: `HMO_RECORD_NOT_FOUND`, `HMO_PLAN_NOT_FOUND`, `HMO_ENROLLMENT_NOT_FOUND`, `EMPLOYEE_NOT_FOUND`. 405: `METHOD_NOT_ALLOWED`.

409: `HMO_SELF_APPROVAL`, `HMO_REQUEST_NOT_PENDING`, `HMO_ENROLLMENT_NOT_PENDING`, `HMO_DEPENDENT_NOT_PENDING`, `HMO_APPROVAL_REQUIRED`, `HMO_ALREADY_SUBMITTED`, `MAXICARE_SUBMISSION_REQUIRED`, `MAXICARE_CONFIRMATION_REQUIRED`, `MAXICARE_REFERENCE_IN_USE`, `MAXICARE_REFERENCE_SHARED` (resend with `allow_shared: true` if Maxicare confirmed it), `HMO_ENROLLMENT_EXISTS`, `HMO_ENROLLMENT_ALREADY_ACTIVE`, `HMO_ENROLLMENT_ALREADY_CLOSED`, `HMO_ENROLLMENT_NOT_ACTIVE`, `HMO_DEPENDENT_ALREADY_ACTIVE`, `HMO_DEPENDENT_ALREADY_CLOSED`, `HMO_PLAN_NOT_ACTIVE`, `HMO_PLAN_DATES_EXCLUDE_ENROLLMENTS`, `HMO_EFFECTIVE_DATE_OUTSIDE_PLAN`, `HMO_EXPIRATION_OUTSIDE_PLAN`, `EMPLOYEE_NOT_ACTIVE`, `EMPLOYEE_HIRE_DATE_REQUIRED`, `HMO_WAITING_PERIOD_NOT_COMPLETED`, `DEPENDENT_DATE_OUTSIDE_ENROLLMENT`, `COMPANY_SPONSORED_DEPENDENT_EXISTS`.

POST `/hmo/{resource}/{id}`

Runs an action on the record in the path.

Access: As above

PATCH `/hmo/{resource}/{id}`

Runs an action, or updates a provider. `PUT` behaves the same.

Access: As above

Providers accept `name` (Maxicare only), `contact_person`, `contact_number` and `status` without an action. Plans, enrollments and dependents need an action (`400` or `409 HMO_ACTION_REQUIRED` otherwise).

PUT `/hmo/{resource}/{id}`

Same as `PATCH`.

Access: As above

DELETE `/hmo/{resource}/{id}`

Always refused: `409 HMO_DELETE_NOT_ALLOWED`.

Access: As above

HMO records are kept; terminate or archive them instead. `history` returns `405`.

13. Loans

Cash advances

New records are cash advances only; SSS and Pag-IBIG loans go in Government Loans.

GET `/loans`

All loan records and their transactions.

Access: `deductions.view`

RESPONSE


```
{
  "data": {
    "loans": [
      {
        "id": "uuid",
        "employeeId": "uuid",
        "employeeNumber": "EMP-0001",
        "employeeName": "Juan Dela Cruz",
        "department": "Engineering",
        "recordType": "Cash Advance",
        "loanType": "Salary Advance",
        "purpose": "Medical expense",
        "principalAmount": 5000,
        "installmentCount": 2,
        "installmentAmount": 2500,
        "repaymentFrequency": "Per Payroll",
        "repaymentStartDate": "2026-11-15",
        "releaseDate": "2026-11-05",
        "approvalStatus": "Approved",
        "approvedBy": "uuid",
        "approvedAt": "2026-11-05T02:00:00.000Z",
        "rejectionReason": null,
        "createdBy": "uuid",
        "createdAt": "2026-11-05T01:00:00.000Z",
        "updatedAt": "2026-11-05T02:00:00.000Z",
        "outstandingBalance": 5000,
        "repaymentsCompleted": 0
      }
    ],
    "transactions": [
      {
        "id": "uuid",
        "loanId": "uuid",
        "eventType": "Approved",
        "amount": 0,
        "note": "HR approved this loan or cash advance for scheduled payroll repayment.",
        "actorId": "uuid",
        "payrollRunId": null,
        "periodStart": null,
        "periodEnd": null,
        "createdAt": "2026-11-05T02:00:00.000Z"
      }
    ]
  }
}
```

GET `/loans/schedule`

Repayments due in a cutoff.

Access: `payroll.view`

QUERY PARAMETERS

Name	Required	Notes
`period_start`, `period_end`	Yes	The cutoff dates.

RESPONSE

```
{
  "data": [
    {
      "loanId": "uuid",
      "employeeId": "uuid",
      "employeeNumber": "EMP-0001",
      "employeeName": "Juan Dela Cruz",
      "recordType": "Cash Advance",
      "loanType": "Salary Advance",
      "amount": 2500,
      "installmentNumber": 1,
      "installmentCount": 2,
      "outstandingAfter": 2500
    }
  ]
}
```

ERRORS

Status	Code	When
400	`INVALID_LOAN_DATE`, `INVALID_PAYROLL_PERIOD`	The dates are invalid.

POST `/loans`

Records a cash advance as Pending. Returns `201`.

Access: `loans.manage`

REQUEST BODY

Field	Required	Notes
`employee_id`	Yes	An active employee.
`record_type`	Yes	Must be `Cash Advance`.
`loan_type`	Yes	`Salary Advance`, `Emergency Cash Advance` or `Other Cash Advance`.
`purpose`	Yes	3 to 500 characters.
`principal_amount`	Yes	More than 0.
`installment_count`	Yes	Whole number, 1 to 1,000.
`release_date`, `repayment_start_date`	Yes	Repayment cannot start before release.

Field	Required	Notes
`repayment_frequency`	Yes	`Per Payroll`, `Monthly First Cutoff` or `Monthly Second Cutoff`.

ERRORS

Status	Code	When
400	`INVALID_LOAN_TYPE`, `INVALID_LOAN_CATEGORY`, `INVALID_LOAN_PURPOSE`, `INVALID_LOAN_AMOUNT`, `INVALID_INSTALLMENT_COUNT`, `INVALID_LOAN_DATE`, `INVALID_REPAYMENT_TERMS`, `INVALID_REPAYMENT_FREQUENCY`	A field is invalid.
404	`EMPLOYEE_NOT_FOUND`	Unknown employee.
409	`EMPLOYEE_NOT_ACTIVE`	The employee is not active.

PATCH `/loans/{id}`

Approves or rejects a Pending record.

Access: `loans.manage`

REQUEST BODY

Field	Required	Notes
`action`	Yes	`approve` or `reject`.
`rejection_reason`	To reject	Up to 500 characters.

ERRORS

Status	Code	When
400	`LOAN_ACTION_NOT_SUPPORTED`, `INVALID_REJECTION_REASON`	A field is invalid.
404	`LOAN_NOT_FOUND`	Unknown record.
409	`LOAN_NOT_PENDING`, `EMPLOYEE_NOT_ACTIVE`	It cannot be reviewed now.

POST `/loans/settle`

Posts the repayments of a Paid run once.

Access: `payroll.release`

With `validate_only`, it runs every check without saving, and also accepts an Exported run. Posting again returns `idempotent: true`.

REQUEST BODY

Field	Required	Notes
`payroll_run_id`	Yes	
`validate_only`	No	`true` to check only.

RESPONSE

```
// Posted
{ "data": { "posted": true, "count": 3, "totalAmount": 7500, "idempotent": false } }

// validate_only
{ "data": { "valid": true, "count": 3, "totalAmount": 7500 } }
```

ERRORS

Status	Code	When
400	`DUPLICATE_LOAN_REPAYMENT`, `INVALID_PAYROLL_PERIOD`, `INVALID_PAYROLL_RUN_DATE`	The run has a repeated installment or invalid dates.
404	`PAYROLL_RUN_NOT_FOUND`	Unknown run.
409	`PAYROLL_NOT_PAID`	Not every employee is Paid.
409	`PAYROLL_LOAN_AMOUNT_MISMATCH`, `PAYROLL_LOAN_TOTAL_MISMATCH`, `PAYROLL_LOAN_SCHEDULE_CHANGED`, `PAYROLL_LOAN_EMPLOYEE_MISMATCH`, `LOAN_APPROVAL_REQUIRED`	The run no longer matches the loan schedule; rebuild it.
409	`PAYROLL_LOAN_REPAYMENT_CONFLICT`, `PAYROLL_PERIOD_ALREADY_SETTLED`	A different batch exists for this run or period.

Government loans

SSS and Pag-IBIG loans encoded from the agency notice. These routes use the HMO permissions (`hmo.view`, `hmo.manage`). Statuses: `Pending`, `Active`, `Rejected`, `Closed`.

GET `/government-loans`

All government loans and their deduction transactions.

Access: `hmo.view`

RESPONSE

```
{
  "data": {
    "loans": [
      {
        "id": "uuid",
        "employeeId": "uuid",
        "employeeNumber": "EMP-0001",
        "employeeName": "Juan Dela Cruz",
        "department": "Engineering",
        "loanType": "SSS",
        "referenceNo": "SSS-12345",
        "totalAmount": 24000,
        "monthlyAmortization": 2000,
        "startDate": "2026-11-01",
        "outstandingBalance": 24000,
        "status": "Active",
        "noticeName": "notice.pdf",
        "remarks": "",
        "rejectionRemarks": null,
```

```
        "verifiedBy": "uuid",
        "verifiedAt": "2026-11-05T02:00:00.000Z",
        "rejectedAt": null,
        "closedAt": null,
        "createdBy": "uuid",
        "createdAt": "2026-11-05T01:00:00.000Z",
        "updatedAt": "2026-11-05T02:00:00.000Z"
      }
    ],
    "transactions": [
      {
        "id": "uuid",
        "loanId": "uuid",
        "payrollRunId": "uuid",
        "periodStart": "2026-11-01",
        "periodEnd": "2026-11-15",
        "amount": 1000,
        "balanceAfter": 23000,
        "postedBy": "uuid",
        "createdAt": "2026-11-30T02:00:00.000Z"
      }
    ]
  }
}
```

GET /government-loans/schedule

Active loans payroll may deduct in a cutoff.

Access: `payroll.view`

QUERY PARAMETERS

Name	Required	Notes
`period_start`, `period_end`	Yes	The cutoff dates.

RESPONSE


```
{
  "data": [
    {
      "loanId": "uuid",
      "employeeId": "uuid",
      "employeeNumber": "EMP-0001",
      "employeeName": "Juan Dela Cruz",
      "loanType": "SSS",
      "referenceNo": "SSS-12345",
      "monthlyAmortization": 2000,
      "outstandingBalance": 24000
    }
  ]
}
```

GET /government-loans/{id}/notice

The attached agency notice.

Access: `hmo.view`

RESPONSE

```
{
  "data": {
    "name": "notice.pdf",
    "data": "data:application/pdf;base64,..."
  }
}
```

ERRORS

Status	Code	When
404	`NOTICE_NOT_FOUND`	No notice for that loan.

POST /government-loans

Encodes a loan as Pending. Returns `201`.

Access: `hmo.manage`

REQUEST BODY

Field	Required	Notes
`employee_id`	Yes	
`loan_type`	Yes	`SSS` or `Pag-IBIG`.
`reference_no`	Yes	Unique among non-rejected loans; up to 100 characters.
`total_amount`, `monthly_amortization`	Yes	More than 0; amortization cannot exceed the total.
`start_date`	Yes	
`notice_name`, `notice_data`	Yes	PDF, PNG or JPG as a base64 data URL, 3 MB or smaller.
`remarks`	No	Up to 1,000 characters.

ERRORS

Status	Code	When
400	`INVALID_LOAN_TYPE`, `INVALID_REFERENCE`, `INVALID_AMORTIZATION`, `INVALID_LOAN_AMOUNT`, `INVALID_LOAN_DATE`, `NOTICE_REQUIRED`, `INVALID_NOTICE`, `NOTICE_TOO_LARGE`	A field is invalid.
404	`EMPLOYEE_NOT_FOUND`	Unknown employee.
409	`DUPLICATE_REFERENCE`	The reference is already used.

PATCH /government-loans/{id}

Edits, verifies or rejects a Pending loan.

Access: `hmo.manage`

REQUEST BODY

Field	Required	Notes
`action`	Yes	`update` (all fields as for creating; the notice is optional), `verify` (becomes Active) or `reject`.
`remarks`	To reject	Up to 1,000 characters.

Returns the loan.

ERRORS

Status	Code	When
400	`LOAN_ACTION_NOT_SUPPORTED`, `INVALID_REMARKS`	A field is invalid.
404	`LOAN_NOT_FOUND`	Unknown loan.
409	`LOAN_NOT_PENDING`	It is no longer Pending.

POST `/government-loans/settle`

Posts the deductions of a Paid run once; a loan reaching zero closes.

Access: `payroll.release`

A run posts completely or not at all. With `validate_only`, it checks without saving, and also accepts an Exported run. A retry also repairs a balance an interrupted attempt left behind.

REQUEST BODY

Field	Required	Notes
`payroll_run_id`	Yes	
`validate_only`	No	`true` to check only.

RESPONSE

```
// Posted
{ "data": { "posted": true, "count": 2, "totalAmount": 2000, "idempotent": false } }

// validate_only
{ "data": { "valid": true, "count": 2, "totalAmount": 2000, "alreadyPosted": 0 } }
```

ERRORS

Status	Code	When
400	`DUPLICATE_LOAN_DEDUCTION`	The same loan appears twice.
404	`PAYROLL_RUN_NOT_FOUND`, `LOAN_NOT_FOUND`	Unknown run or loan.
409	`PAYROLL_NOT_PAID`	Not every employee is Paid.
409	`GOVERNMENT_LOAN_AMOUNT_MISMATCH`, `GOVERNMENT_LOAN_EMPLOYEE_MISMATCH`, `LOAN_NOT_ACTIVE`, `LOAN_BALANCE_EXCEEDED`	The run no longer matches the loans; rebuild it.

14. Claims and reimbursement

Employees submit claims through ESS. PBMS users approve, return or reject them by saving `claims:records` (see Synchronized app data), then record the payment here. Money is sent outside PBMS. Claim objects use camelCase fields such as `claimNumber`, `approvalStatus`, `paymentStatus`, `paymentMethod`, `paymentReference` and `paymentDate`.

GET `/claims/totals`

Claim totals by month, type and status, with no employee or receipt details.

Access: `claims.view` or `payroll.view`

RESPONSE

```
{
  "data": [
    {
      "month": "2026-10",
      "claimType": "Transportation",
      "status": "Paid",
      "count": 4,
      "amount": 1802
    }
  ]
}
```

GET `/claims/{id}/receipt`

A claim's receipt file.

Access: `claims.view`

RESPONSE

```
{
  "data": {
    "name": "taxi-receipt.jpg",
    "mime_type": "image/jpeg",
    "data_url": "..."
  }
}
```

ERRORS

Status	Code	When
404	`RECEIPT_NOT_FOUND`	No receipt on file.

GET `/claims/{id}/payment-proof`

A claim's proof of payment, or its batch's proof.

Access: `claims.view`

RESPONSE

```
{
  "data": {
    "name": "bdo-confirmation.pdf",
    "mime_type": "application/pdf",
```

```
    "data_url": "data:application/pdf;base64,..."
  }
}
```

ERRORS

Status	Code	When
404	`PROOF_NOT_FOUND`	No proof attached.

POST `/claims/{id}/payment`

Records the reimbursement of one approved claim.
Access: `claims.pay`

REQUEST BODY

Field	Required	Notes
`method`	Yes	`Bank Transfer`, `Cash` or `E-wallet`.
`reference`	Yes	Transaction or cash voucher number, 3 to 100 characters.
`date`	Yes	Not in the future, and not before the expense date.
`proof`	For cash	`{ name, mime_type, data }`: JPEG, PNG or PDF as base64, 3 MB or smaller.

EXAMPLE REQUEST

```
{
  "method": "Bank Transfer",
  "reference": "BDO-778899",
  "date": "2026-10-27"
}
```

Returns the updated claim. The employee gets an ESS notification.

ERRORS

Status	Code	When
400	`CLAIM_PAYMENT_REFUSED`	A field is invalid. The message says which.
403	`CLAIM_PAYMENT_REFUSED`	It is your own claim, or you approved it.
404	`CLAIM_NOT_FOUND`	Unknown claim.
409	`CLAIM_PAYMENT_REFUSED`	The claim is not approved, or is already reimbursed.
409	`CLAIM_IN_BATCH`	The claim is in a batch: mark it paid or cancel it.

POST `/claims/batches`

Creates a batch of approved claims paid by one bank transfer. Returns `201`.
Access: `claims.pay`

REQUEST BODY

Field	Required	Notes
`claim_ids`	Yes	1 to 500 approved, unpaid claims not in another batch.
`pay_date`	Yes	Today or later.

RESPONSE

```
{
  "data": {
    "id": "rb-20261105010000-ab12cd",
    "number": "RB-2026-11-001",
    "status": "Exported",
    "payDate": "2026-11-06",
    "claimIds": [
      "uuid"
    ],
    "total": 450.5,
    "createdAt": "2026-11-05T01:00:00.000Z",
    "createdBy": "Maria Santos",
    "createdById": "uuid"
  }
}
```

ERRORS

Status	Code	When
400 / 403 / 404 / 409	`BATCH_REFUSED`	A claim or the date is not allowed. The message says why.

POST `/claims/batches/{id}/payment`

Marks a batch paid after the bank transfer.

Access: `claims.pay`

Claims the bank did not pay go back to the queue.

REQUEST BODY

Field	Required	Notes
`reference`	Yes	Bank reference, 3 to 100 characters.
`date`	Yes	Not in the future, and not before any expense date.
`failed_claim_ids`	No	Claims in the batch that were not paid. If all failed, cancel the batch instead.
`proof`	No	`{ name, mime_type, data }` as above.

Returns the batch with `status: "Paid"`, `paidClaimIds`, `failedClaimIds`, `paidTotal`, `reference`, `paymentDate`, `paidAt`, `paidBy`, `paidById` and `hasProof`.

ERRORS

Status	Code	When
400 / 403 / 404 / 409	`BATCH_PAYMENT_REFUSED`	The message says why.

POST `/claims/batches/{id}/cancel`

Cancels an unpaid batch; its claims return to the queue.

Access: `claims.pay`

REQUEST BODY

Field	Required	Notes
`reason`	Yes	At least 3 characters.

ERRORS

Status	Code	When
404	`BATCH_NOT_FOUND`	Unknown batch.
409	`BATCH_CLOSED`	Already paid or cancelled.

15. Integration: HR2 attendance

HR2 sends daily attendance to PBMS. Send at least once before each cutoff closes (the 15th and the end of the month), with `synced_through` set to the cutoff's last day. PBMS never calls HR2.

POST `/integrations/hr2/attendance`

Receives attendance records.

Access: `X-API-Key` (HR2)

One record per employee per date: sending the same pair again replaces the earlier record, so corrections are made by resending. Send every work day, including `Absent` and `Day Off`; a work day with no record is not paid. Valid records are saved even when others in the request are rejected.

REQUEST BODY

Field	Required	Notes
`synced_through`	Recommended	Last date HR2 has finished. It never moves backwards. If omitted, the latest record date counts.
`records`	Yes	At most 5,000 per request.
`records[].employee_no`	Yes	As in PBMS. Case and surrounding spaces are ignored.
`records[].date`	Yes	`YYYY-MM-DD`.
`records[].status`	Yes	`Present`, `Absent`, `On Leave` or `Day Off`.
`records[].time_in`, `time_out`	For `Present`	24-hour `HH:MM` or null. A `Present` day without both is not paid.
`records[].worked_hours`	No	0 to 24, or null.
`records[].late_min`, `undertime_min`, `ot_min`	No	Whole minutes, 0 to 1,440; default 0. Used as received.
`records[].ot_status`	When `ot_min` > 0	`Approved`, `Pending`, `Rejected` or null. Only `Approved` overtime is paid.

Field	Required	Notes
`records[].leave_type`	No	For `On Leave` days. Up to 100 characters.

EXAMPLE REQUEST

```
{
  "synced_through": "2026-10-15",
  "records": [
    {
      "employee_no": "EMP-0001",
      "date": "2026-10-01",
      "time_in": "08:12",
      "time_out": "17:30",
      "worked_hours": 8.3,
      "late_min": 12,
      "undertime_min": 0,
      "ot_min": 30,
      "status": "Present",
      "leave_type": null,
      "ot_status": "Approved"
    }
  ]
}
```

RESPONSE

```
{
  "data": {
    "received": 118,
    "rejected": [
      {
        "index": 7,
        "employee_no": "EMP-9999",
        "date": "2026-10-03",
        "reason": "employee_no is not a PBMS employee"
      }
    ],
    "synced_through": "2026-10-15",
    "total_records": 2240
  }
}
```

ERRORS

Status	Code	When
400	`VALIDATION_ERROR`	`records` is not a list, or `synced_through` is not a date.
413	`TOO_MANY_RECORDS`	More than 5,000 records.
503	`HR2_NOT_CONFIGURED`	`HR2_API_KEY` is not set.

```
curl -X POST "https://<pbms-render-host>/integrations/hr2/attendance" \
  -H "Content-Type: application/json" \
  -H "X-API-Key: $PBMS_HR2_API_KEY" \
  -d '{"synced_through":"2026-10-15","records":[...]}'
```

16. Integration: Finance

Finance reads submitted runs: their figures, decision, funding and payout progress. Decisions are made in PBMS, so this API is read-only. Runs still being prepared (Draft, Computed) are not listed.

GET `/integrations/finance/payroll-runs`

Lists submitted runs, newest first.

Access: `X-API-Key` (Finance)

QUERY PARAMETERS

Name	Required	Notes
`status`	No	`pending`, `returned`, `rejected`, `approved`, `funded`, `exported`, `paid` or `closed`.

RESPONSE


```
{
  "data": [
    {
      "id": "5f1c...",
      "run_type": "regular",
      "period_from": "2026-10-01",
      "period_to": "2026-10-15",
      "pay_date": "2026-10-15",
      "status": "exported",
      "employees": 42,
      "totals": {
        "gross": 1250000,
        "deductions": 210500.5,
        "net": 1039499.5,
        "withholding_tax": 85210.25
      },
      "prepared_by": "HR User",
      "submitted_at": "2026-10-13T02:10:00.000Z",
      "decided_by": "Admin User",
      "decided_at": "2026-10-13T05:00:00.000Z",
      "decision_comment": null,
      "funded_at": "2026-10-14T01:00:00.000Z",
      "funding_amount": 1039499.5,
      "funding_receipt": {
        "file_name": "bdo-transfer.pdf",
        "mime_type": "application/pdf",
        "size_bytes": 182340,
        "uploaded_by": "Admin User",
        "uploaded_at": "2026-10-14T01:00:00.000Z"
      },
      "payouts": {
        "pending": 0,
        "exported": 40,
        "paid": 1,
        "failed": 1
      },
      "paid_at": null,
      "closed_at": null
    }
  ]
}
```

ERRORS

Status	Code	When
400	`VALIDATION_ERROR`	Invalid `status`.
503	`FINANCE_NOT_CONFIGURED`	`FINANCE_API_KEY` is not set.

GET `/integrations/finance/payroll-runs/{id}`

One run with its employees.
Access: `X-API-Key` (Finance)

The same fields plus `rows`: `employee_no`, `name`, `gross`, `deductions`, `net`, `withholding_tax`, `sss`, `philhealth`, `pagibig`, `employer_sss`, `employer_philhealth`, `employer_pagibig`, `payout_method`, `payout_status` and `payout_failure_reason`.

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`	No submitted run with that id.

GET `/integrations/finance/payroll-runs/{id}/funding-receipt`

The funding receipt file (JPG, PNG or PDF).
Access: `X-API-Key` (Finance)

QUERY PARAMETERS

Name	Required	Notes
`download`	No	`1` downloads it instead of showing it inline.

ERRORS

Status	Code	When
404	`PAYROLL_RUN_NOT_FOUND`, `RECEIPT_NOT_FOUND`	Unknown run, or it has no receipt.

POST `/integrations/finance/payroll-runs/{id}/decision`

Removed: always returns `410 DECISIONS_IN_PBMS`.
Access: `X-API-Key` (Finance)

17. Integration: ESS

ESS sends employees' claims and correction requests to PBMS, and reads back statuses, payslips and notifications. PBMS never calls ESS. Every route needs the ESS key, and returns `503 ESS_NOT_CONFIGURED` until `ESS_API_KEY` is set. For the routes that take `employee_no`: `400 VALIDATION_ERROR` when it is missing and `404 EMPLOYEE_NOT_FOUND` when unknown.

POST `/integrations/ess/claims`

Submits a claim. Returns `201`, or `200` for a repeated `ess_reference`.
Access: `X-API-Key` (ESS)

An employee who has left can submit for 30 days after leaving.

REQUEST BODY

Field	Required	Notes
`employee_no`	Yes	As in PBMS.
`claim_type`	Yes	`Food`, `Transportation` or `Office Supplies`.
`amount`	Yes	More than 0, at most 1,000,000, up to 2 decimals.
`date`	Yes	Not in the future (Philippine time).
`business_purpose`	Yes	5 to 1,000 characters.
`project_site`	No	Up to 200 characters.
`receipt`	Yes	`name`, `mime_type` (`image/jpeg`, `image/png` or `application/pdf`) and `data` (base64, 3 MB or smaller).
`certified`	Yes	Must be `true`: the employee confirms a business expense.
`ess_reference`	Recommended	ESS's own id, up to 100 characters. Sending it again returns the claim already received.

EXAMPLE REQUEST

```
{
  "employee_no": "EMP-0001",
  "claim_type": "Transportation",
  "amount": 450.5,
  "date": "2026-10-20",
  "business_purpose": "Taxi to the Makati client site for the site inspection.",
  "project_site": "Makati Tower",
  "receipt": {
    "name": "taxi-receipt.jpg",
    "mime_type": "image/jpeg",
    "data": "<base64>"
  },
  "certified": true,
  "ess_reference": "ESS-CLM-88412"
}
```

Returns the claim, shaped as in the list below.

ERRORS

Status	Code	When
403	`EMPLOYEE_NOT_ACTIVE`	The employee left more than 30 days ago.

PUT `/integrations/ess/claims/{claimNumber}`

Resubmits a Returned claim, with all fields again. It goes back to Pending Approval.

Access: `X-API-Key` (ESS)

ERRORS

Status	Code	When
403	`EMPLOYEE_NOT_ACTIVE`	As above.
404	`CLAIM_NOT_FOUND`	No such claim for that employee.
409	`CLAIM_NOT_RETURNED`	Only a Returned claim can be resubmitted.

GET `/integrations/ess/claims`

An employee's claims, newest first.

Access: `X-API-Key` (ESS)

QUERY PARAMETERS

Name	Required	Notes
`employee_no`	Yes	

RESPONSE


```
{
  "data": [
    {
      "claim_number": "CLM-2026-0042",
      "claim_type": "Transportation",
      "amount": 450.5,
      "date": "2026-10-20",
      "business_purpose": "Taxi to the Makati client site for the site inspection.",
      "project_site": "Makati Tower",
      "receipt_name": "taxi-receipt.jpg",
      "status": "Returned",
      "notes": "The receipt is unreadable; attach a clearer photo.",
      "payment_method": null,
      "payment_date": null,
      "payment_reference": null,
      "submitted_at": "2026-10-20T03:12:00.000Z",
      "updated_at": "2026-10-21T01:05:00.000Z",
      "ess_reference": "ESS-CLM-88412"
    }
  ]
}
```


`status`: `Pending Approval`, `Returned`, `Rejected`, `Approved` or `Reimbursed` (`Included in Payroll` only on older claims). `payment_method`: `Bank Transfer`, `Cash` or `E-wallet` (`Paid Separately` on older claims).

GET `/integrations/ess/payslips`

An employee's released payslips, newest release first.

Access: `X-API-Key` (ESS)

QUERY PARAMETERS

Name	Required	Notes
`employee_no`	Yes	

RESPONSE


```
{
  "data": [
    {
      "runId": "4f0c...",
      "period": "2026-10-01 to 2026-10-15",
      "from": "2026-10-01",
      "to": "2026-10-15",
      "payDate": "2026-10-15",
      "empId": "EMP-0001",
      "name": "Ana Reyes",
      "position": "Site Engineer",
      "basic": 15000,
      "allowance": 500,
      "bonuses": 0,
      "overtime": 360.58,
      "holidayLeave": 0,
      "gross": 15860.58,
      "absences": 0,
      "lateUt": 144.23,
      "sss": 750,
      "philhealth": 375,
      "pagibig": 100,
      "hmo": 250,
      "tax": 611.15,
      "loans": 0,
      "other": 0,
      "totalDeductions": 2230.38,
      "net": 13630.2,
      "employmentStatus": "Regular",
      "earnings": [
        {
          "label": "Basic Salary",
          "amount": 15000
        }
      ],
      "deductionGroups": [
        {
          "title": "Statutory Contributions",
          "lines": [
            {
              "label": "SSS Contribution",
              "amount": 750
            }
          ],
          "subtotal": 750
        }
      ],
      "releasedAt": "2026-10-15T08:30:00.000Z"
    }
  ]
}
```


`earnings` add up to `gross`; group `subtotal`s add up to `totalDeductions`. Groups appear in this order, only with an amount: Statutory Contributions, Withholding Tax, Attendance Deductions, Other Deductions. Payslips released before October 31, 2026 used earlier group names; those released before October 29, 2026 have no `employmentStatus`, `earnings` or `deductionGroups`, so show their register figures.

GET /integrations/ess/notifications

An employee's latest 100 notifications, newest first.

Access: `X-API-Key` (ESS)

QUERY PARAMETERS

Name	Required	Notes
`employee_no`	Yes	

RESPONSE

```
{
  "data": [
    {
      "id": "9b1e...",
      "title": "Payslip available",
      "message": "Your payslip for 2026-10-01 to 2026-10-15 is ready. Net pay ₱13,630.20.",
      "run_id": "4f0c...",
      "created_at": "2026-10-15T08:30:00.000Z",
      "read_at": null
    }
  ]
}
```

Titles: `Payslip available`, `Claim approved`, `Claim returned for correction`, `Claim rejected`, `Claim reimbursed`. `run_id` is set only for payslips. ESS keeps its own read state.

POST /integrations/ess/corrections

Requests a payroll correction. Returns `201`, or `200` for a repeated `ess_reference`.

Access: `X-API-Key` (ESS)

REQUEST BODY

Field	Required	Notes
`employee_no`	Yes	
`run_id`	Yes	The payslip's `runId`: an approved run that includes the employee.
`request`	Yes	What is wrong, 5 to 1,000 characters.
`amount`	Yes	More than 0, at most 1,000,000, up to 2 decimals.
`ess_reference`	Recommended	ESS's own id, up to 100 characters.

EXAMPLE REQUEST

```
{
  "employee_no": "EMP-0001",
  "run_id": "c386ed75-...",
  "request": "Overtime on Oct 20 (2 hours) is missing from my Oct 30 payslip.",
  "amount": 450.5,
  "ess_reference": "ESS-COR-1042"
}
```

Returns the correction, shaped as in the list below.

GET `/integrations/ess/corrections`

An employee's corrections, newest first.

Access: `X-API-Key` (ESS)

QUERY PARAMETERS

Name	Required	Notes
`employee_no`	Yes	

RESPONSE

```
{
  "data": [
    {
      "id": "uuid",
      "correction_number": "COR-2026-0001",
      "employee_no": "EMP-0001",
      "employee_name": "Juan Dela Cruz",
      "run_id": "c386ed75-...",
      "period": "2026-10-10 to 2026-10-25",
      "request": "Overtime on Oct 20 (2 hours) is missing from my Oct 30 payslip.",
      "amount": 450.5,
      "status": "Rejected",
      "decision_comment": "The overtime was paid in the Oct 15 run.",
      "decided_by": "Finance User",
      "decided_at": "2026-10-27T02:10:00.000Z",
      "completed_by": null,
      "completed_at": null,
      "completion_note": null,
      "submitted_at": "2026-10-26T01:00:00.000Z",
      "ess_reference": "ESS-COR-1042"
    }
  ]
}
```

18. Health checks and configuration

GET `/health`

Confirms the service is running.

Access: Public

RESPONSE

```
{
  "ok": true
}
```

GET `/api/health`

Row counts, to check which database the service reads.

Access: Public

RESPONSE

```
{
  "ok": true,
  "database": "pbms",
  "employees": 45,
  "attendance": 2240
}
```

Environment variables

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string.
`JWT_SECRET`	Signs bearer tokens.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Create the first Admin when the users table is empty.
`ALLOWED_ORIGINS`	Comma-separated browser origins allowed by CORS.
`HR2_API_KEY`, `FINANCE_API_KEY`, `ESS_API_KEY`	Integration keys, at least 32 characters each.